

AI SMART MONEY TRADING SYSTEM

Spesifikasi Fitur & Prompt Agentic AI

Panduan Implementasi Lengkap untuk AI Coding Agent

Proyek	AI Smart Money Trading System (Lastico / Hype)
Dokumen	Feature Specification & Agentic AI Implementation Guide
Versi	v2 — Melengkapi Dokumentasi Arsitektur v2 & Lampiran Teknis
Tanggal	21 Juni 2026
Referensi	hype_dokumentasi_v2.pdf + hype_dokumentasi_v2_1.pdf

Dokumen ini adalah output analisis fitur berdasarkan Dokumentasi Arsitektur v2 dan Lampiran Teknis. Setiap fitur dilengkapi dengan: tujuan bisnis, gambaran sistem, alur bisnis, skenario penggunaan, dan Prompt Agentic AI yang siap diberikan kepada AI Coding Agent untuk implementasi.

DAFTAR ISI

F-01	Wallet Movement Monitor	Layer 0 — Data Source
F-02	Rule-Based Relevance Filter	Layer 0/1 — Filter Engine
F-03	Wallet Movement Trigger Engine	Layer 1 — Trigger Engine
F-04	Feature Extraction Pipeline	Layer 2 — ML Pipeline
F-05	XGBoost Inference Engine	Layer 2 — ML Pipeline
F-06	Token Safety Check Gate	Layer 2/3 — Safety Gate
F-07	Web Dashboard Alert Engine	Layer 3 — Decision & Display
F-08	Auto Trade Execution	Layer 3 — Eksekusi Otomatis
F-09	Three-Layer Position Protection	Layer 3 — Proteksi Posisi
F-10	24h Retrain & Rollback Scheduler	Layer 4 — Adaptive Learning
F-11	Model v0 Bootstrap	Layer 4 — Cold Start
F-12	Dynamic Wallet Discovery	Layer 0 — Enhancement B1
F-13	Token Age & Liquidity Hard Filter	Layer 1 — Enhancement B2
F-14	Dedup & Cooldown per Wallet-Token	Layer 1 — Enhancement B3
F-15	RPC Fallback & Reconnect	Layer 0 — Enhancement B4
F-16	Position Correlation Cap	Eksekusi — Enhancement B5
F-17	State Persistence & Crash Recovery	Cross-Layer — A6
F-18	Centralized Config Management	Cross-Layer — A4
F-19	Error & Recovery Flow Engine	Cross-Layer — A3

FITUR 01

WALLET MOVEMENT MONITOR

Memantau transaksi on-chain Whale A & B secara real-time 24/7

F-01 · WALLET MOVEMENT MONITOR

Sistem Pemantau Pergerakan Wallet Whale

Tujuan Fitur

Mengapa Diperlukan

- Seluruh sistem bergantung pada deteksi pergerakan 2 wallet whale (Whale A & Whale B) sebagai sinyal awal.
- Tanpa pemantauan real-time, sistem tidak akan tahu kapan whale melakukan swap, transfer, atau perubahan LP.
- Pemantauan harus 24/7 karena pasar meme coin aktif kapan saja, termasuk dini hari.

Siapa yang Menggunakan

- Sistem internal (Layer 0) — tidak ada interaksi manual dari pengguna.
- Developer memantau status koneksi WebSocket lewat dashboard.

Gambaran Fitur

Fitur ini adalah fondasi dari seluruh sistem. WebSocket connection dibuka ke blockchain RPC/Indexer dan melakukan subscription ke aktivitas on-chain dari dua wallet target. Setiap event yang masuk (swap, transfer, LP change) direkam dengan timestamp dan diteruskan ke lapisan berikutnya. Koneksi dijaga tetap aktif dengan mekanisme heartbeat/ping, dan jika terputus akan otomatis reconnect.

Data yang Diproses

Field	Tipe	Keterangan
wallet_address	TEXT	Whale A atau Whale B yang memicu event
event_type	TEXT	swap / transfer / lp_change
token_mint	TEXT	Contract address token yang terlibat
amount_usd	REAL	Estimasi nilai transaksi dalam USD
signature	TEXT	Transaction signature on-chain (unique)
timestamp_utc	DATETIME	Waktu event terjadi di blockchain

Alur Bisnis

- 1

Inisialisasi Koneksi

Saat sistem startup, buka WebSocket ke RPC provider dan subscribe ke wallet Whale A & B.
- 2

Dengarkan Event

Sistem idle mendengarkan event on-chain secara real-time (logsSubscribe / accountSubscribe).
- 3

Event Masuk

Setiap transaksi dari wallet target masuk sebagai raw event dengan signature, instruction type, token mint, jumlah.

4 Heartbeat Check

Setiap N detik, sistem mengirim ping ke RPC. Jika tidak ada respons, tandai koneksi sebagai tidak sehat.

5 Reconnect Otomatis

Jika koneksi terputus, lakukan exponential backoff reconnect. Jika gagal > N kali, alihkan ke RPC secondary (F-15).

6 Teruskan ke Filter

Event yang berhasil ditangkap diteruskan ke Rule-Based Relevance Filter (F-02).

Skenario Penggunaan

Skenario 1 — Whale A Melakukan Swap

Tujuan: Mendeteksi aktivitas trading whale secara real-time.

Langkah: Whale A swap 10 SOL ke token XYZ. RPC node menerima transaksi → event masuk ke sistem → diteruskan ke F-02.

Hasil: Sistem meneruskan event ke filter. Jika relevan, trigger engine mulai memantau window 5 menit.

Skenario 2 — Koneksi RPC Terputus Saat Posisi Terbuka

Tujuan: Memastikan monitoring tidak berhenti walau koneksi putus.

Langkah: RPC primary timeout. Sistem gagal ping → deteksi koneksi putus → exponential backoff retry → setelah 3x gagal, switch ke RPC secondary.

Hasil: Monitoring posisi tetap berjalan via RPC secondary. Notifikasi switchover muncul di dashboard.

PROMPT AGENTIC AI — F-01

Anda adalah Senior Software Engineer. Bangun fitur WALLET MOVEMENT MONITOR.

Tujuan fitur:

Memantau pergerakan transaksi on-chain dua wallet (Whale A & Whale B) secara real-time 24/7 menggunakan WebSocket subscription ke Solana RPC/Indexer, dan meneruskan setiap event ke lapisan filter berikutnya.

Fungsi utama:

- Membuka dan menjaga koneksi WebSocket ke Solana RPC provider (logsSubscribe / accountSubscribe)
- Subscribe ke dua wallet address yang dikonfigurasi di config.yaml (watchlist_wallets)
- Menangkap event: swap DEX, transfer token, perubahan LP
- Parsing event menjadi struktur data: wallet_address, event_type, token_mint, amount_usd, signature, timestamp_utc
- Mekanisme heartbeat: ping setiap N detik, deteksi koneksi tidak sehat jika tidak ada respons
- Exponential backoff reconnect saat koneksi terputus
- Integrasi dengan RPC Fallback (F-15) jika reconnect gagal > N kali
- Meneruskan event ke Rule-Based Relevance Filter (F-02) via internal event queue

Alur bisnis:

1. Startup: baca watchlist_wallets dari database, buka koneksi WS ke RPC primary
2. Subscribe ke setiap wallet address di watchlist
3. Loop: terima event → parse → validasi signature unik → kirim ke queue F-02
4. Paralel: heartbeat goroutine/task memeriksa koneksi setiap 30 detik
5. Jika koneksi putus: trigger reconnect dengan backoff (1s, 2s, 4s, 8s...) max 5x
6. Jika reconnect gagal semua: emit event ke F-15 (RPC Fallback), log ke error_log

Validasi:

- Wallet address harus valid Solana pubkey format (base58, 32-44 chars)
- Event signature harus unik (dedup dalam memory window 60 detik)
- Timestamp harus dalam range reasonable (tidak lebih dari 60 detik dari now)
- Hanya proses event dari wallet yang ada di watchlist aktif (active=TRUE)

Hak akses: Komponen internal sistem — tidak ada user-facing access. Konfigurasi wallet hanya bisa diubah melalui F-18.

Kondisi gagal:

- RPC tidak tersedia saat startup: retry 3x, kemudian fallback ke RPC secondary
- Event tidak bisa di-parse: log ke error_log dengan raw data, skip event, lanjut
- Queue F-02 penuh: implementasi backpressure, slow down ingestion

Integrasi:

- Input: watchlist_wallets dari database (F-12 untuk dynamic discovery)
- Output: event stream ke F-02 (Rule-Based Filter)
- Koneksi: F-15 (RPC Fallback) jika primary gagal
- Monitoring: F-07 (Dashboard) untuk status koneksi

Pastikan implementasi:

- Clean Architecture, SOLID Principle, Modular, Mudah dikembangkan, Mudah diuji, Production Ready

Sebelum membuat kode, lakukan analisis kebutuhan terlebih dahulu dan identifikasi seluruh komponen yang diperlukan.

FITUR 02

RULE-BASED RELEVANCE FILTER

Memisahkan event trading relevan dari noise (transfer internal, top-up gas)

F-02 · RULE-BASED RELEVANCE FILTER

Filter Relevansi Berbasis Aturan

Tujuan Fitur

Masalah yang Diselesaikan

- Tidak semua pergerakan wallet adalah sinyal trading. Transfer gas, pindah ke wallet sendiri, atau top-up exchange bukan sinyal yang relevan.
- Jika event tidak relevan ikut masuk ke trigger window, dataset training akan tercemar dengan false signal.
- Filter ini adalah 'pintu masuk pertama' — memastikan hanya event bermakna secara trading yang diproses lebih jauh.

Gambaran Fitur

Setiap event dari F-01 divalidasi berdasarkan aturan: apakah transaksi ini melalui DEX router yang dikenal? Apakah nilainya di atas threshold minimum? Apakah bukan transfer ke wallet sendiri atau custodial exchange? Apakah bukan sekadar top-up gas (SOL transfer kecil)? Event yang lolos diteruskan ke Trigger Engine (F-03). Yang gagal di-log sebagai 'irrelevant' untuk audit namun tidak masuk training.

Aturan Filter (Rule Set)

Aturan	Kondisi LOLOS	Kondisi GAGAL
DEX Router Check	Interaksi dengan alamat router DEX publik yang dikenal (Raydium, Orca, etc)	Transfer langsung ke wallet default / tanpa DEX router
Value Threshold	Nilai transaksi > threshold minimum (dikonfigurasi di config)	Transaksi nilai sangat kecil (top-up gas, dust)
Self-Transfer Check	Pengirim dan penerima adalah wallet berbeda (bukan transfer sendiri)	Transfer ke wallet sendiri / rebalancing internal
Exchange Address	Bukan transfer ke custodial exchange (Binance, Coinbase, etc)	Transfer ke wallet centralized exchange
LP Change	LP add/remove yang signifikan	Perubahan LP trivial di bawah threshold

Alur Bisnis

- 1

Terima Event

Menerima event dari F-01 melalui internal queue.
- 2

Cek DEX Router

Bandingkan instruction accounts dengan daftar DEX router yang dikenal. Jika tidak cocok → log 'irrelevant_non_dex', skip.
- 3

Cek Value Threshold

Hitung estimasi nilai USD. Jika di bawah threshold config → log 'irrelevant_low_value', skip.
- 4

Cek Self-Transfer

Verifikasi penerima bukan wallet sendiri dari watchlist. Jika ya → log 'irrelevant_self_transfer', skip.
- 5

Cek Exchange Address

Verifikasi penerima bukan alamat custodial exchange yang dikenal. Jika ya → skip.

6**Lolos Semua Filter**

Event diteruskan ke F-03 (Trigger Engine) via queue dengan flag 'relevant'.

7**Log Semua Hasil**

Baik lolos maupun gagal, semua event di-log dengan alasan untuk audit trail.

PROMPT AGENTIC AI — F-02

Anda adalah Senior Software Engineer. Bangun fitur RULE-BASED RELEVANCE FILTER.

Tujuan fitur:

Memfilter event on-chain dari F-01 menggunakan aturan deterministik untuk memisahkan aktivitas trading yang relevan dari noise (transfer internal, top-up gas, transfer ke custodial exchange).

Fungsi utama:

- Menerima event dari F-01 via internal async queue
- Menjalankan serangkaian rule check secara berurutan (fail-fast: stop di aturan pertama yang gagal)
- Meneruskan event lolos ke F-03 (Trigger Engine)
- Menyimpan log semua event (lolos & gagal) dengan alasan untuk audit
- Daftar DEX router address dan exchange address dapat dikonfigurasi (tidak hardcoded)

Alur bisnis:

1. Dequeue event dari F-01
2. Rule 1: Periksa apakah transaksi melibatkan DEX router yang dikenal → jika tidak, log 'irrelevant_non_dex', skip
3. Rule 2: Estimasi nilai USD → jika < threshold config, log 'irrelevant_low_value', skip
4. Rule 3: Periksa self-transfer (sender == receiver dalam watchlist) → jika ya, skip
5. Rule 4: Periksa receiver bukan custodial exchange address → jika ya, skip
6. Rule 5: Untuk LP change, periksa apakah perubahan signifikan (> threshold LP)
7. Jika semua lolos: enqueue ke F-03 dengan metadata relevance_reason

Validasi:

- Daftar DEX router harus dimuat dari konfigurasi terpusat (config.yaml via F-18), bukan hardcoded
- Semua filtering harus idempotent (hasil sama untuk input sama)
- Threshold nilai USD dikonfigurasi di config.yaml

Hak akses: Komponen internal, tidak ada user-facing access.

Kondisi gagal:

- Daftar DEX router kosong: emit warning ke dashboard, gunakan fallback minimal list, lanjut operasi
- Event malformed: log error dengan raw event, skip, jangan crash

Integrasi:

- Input: event queue dari F-01
- Output: filtered event queue ke F-03

- Konfigurasi: F-18 (config.yaml) untuk DEX router list, threshold
- Audit: simpan filter_log ke SQLite untuk debugging

Pastikan implementasi: Clean Architecture, SOLID Principle, Modular, Mudah dikembangkan, Mudah diuji, Production Ready

Sebelum membuat kode, lakukan analisis kebutuhan terlebih dahulu dan identifikasi seluruh komponen yang diperlukan.

FITUR 03

WALLET MOVEMENT TRIGGER ENGINE

Window 5 menit AND/OR — memutuskan kapan pipeline ML dijalankan

F-03 · WALLET MOVEMENT TRIGGER ENGINE

Mesin Pemicu Berbasis Window Waktu

Tujuan Fitur

Konsep Kunci

- Mode AND: sistem baru 'yakin' ada sinyal jika KEDUA whale (A dan B) sama-sama bergerak ke token yang sama dalam window 5 menit.
- Mode OR: cukup salah satu whale bergerak untuk memicu analisis.
- Window ini mencegah satu aktivitas acak satu whale langsung memicu trade — ada konfirmasi yang diperlukan.
- Mode AND/OR dikonfigurasi di config.yaml dan bisa diubah tanpa menyentuh kode.

Gambaran Fitur

Setelah event lolos filter relevansi (F-02), Trigger Engine membuka 'window observasi' 5 menit. Selama window ini, sistem mengumpulkan event dari wallet lain. Pada akhir window, sistem mengevaluasi apakah kondisi trigger terpenuhi (AND/OR). Jika ya, system 'fire event' dan mengirim sinyal ke pipeline ML (F-04). Jika tidak, window di-reset dan monitoring dilanjutkan.

Alur Bisnis

- 1

Terima Event Relevan

Event relevan masuk dari F-02. Catat token_mint, wallet_source, timestamp.
- 2

Buka/Perbarui Window

Jika tidak ada window aktif untuk token ini, buka window baru (5 menit countdown). Jika sudah ada, tambahkan event ke window.
- 3

Cek Dedup & Cooldown

Sebelum menambahkan, periksa cooldown state (F-14) — jika (wallet, token) masih dalam cooldown, event diabaikan untuk trigger baru.
- 4

Evaluasi Kondisi

Mode AND: tunggu sampai window selesai, cek apakah Whale A DAN Whale B sama-sama ada. Mode OR: langsung fire begitu satu whale muncul.
- 5

Fire Event

Jika kondisi terpenuhi: buat trigger_event dengan semua metadata, kirim ke F-04 (Feature Extraction).
- 6

Reset / Lanjut

Jika kondisi tidak terpenuhi: reset window, log sebagai 'trigger_missed', lanjut monitoring.

PROMPT AGENTIC AI — F-03

Anda adalah Senior Software Engineer. Bangun fitur WALLET MOVEMENT TRIGGER ENGINE.

Tujuan fitur:

Menentukan kapan pipeline ML harus dijalankan berdasarkan evaluasi window waktu 5 menit (configurable)

terhadap event dari dua wallet target. Mode AND/OR dikonfigurasi dan dapat diubah tanpa deploy ulang.

Fungsi utama:

- Menerima filtered event dari F-02
- Mengelola 'trigger windows' per token (dictionary: token_mint -> window_state)
- Mode AND: fire jika KEDUA wallet A dan B muncul dalam window
- Mode OR: fire jika salah satu wallet muncul dalam window
- Integrasi dengan dedup/cooldown check (F-14) sebelum menambah event ke window
- Fire trigger_event ke F-04 jika kondisi terpenuhi
- Auto-expire window jika timeout tanpa kondisi terpenuhi

Alur bisnis:

1. Terima event (token_mint, wallet_source, timestamp, raw_event_data)
2. Cek cooldown (F-14): jika (wallet_source, token_mint) dalam cooldown → ignore for new trigger
3. Cek apakah ada window aktif untuk token_mint ini
4. Jika tidak ada: buat window baru, set expiry = now + window_minutes (dari config)
5. Tambahkan event ke window state
6. Evaluasi kondisi: AND → $\text{set}(\text{wallets_in_window}) == \{\text{whale_a}, \text{whale_b}\}$ | OR → $\text{len}(\text{wallets_in_window}) \geq 1$
7. Jika kondisi terpenuhi: create trigger_event, enqueue ke F-04, set cooldown (F-14), close window
8. Background task: setiap 30 detik, expire window yang sudah melewati timeout, log 'window_expired'

Validasi:

- window_minutes harus dari config (default: 5), bukan hardcoded
- Mode AND/OR harus dari config.yaml field trigger_engine.mode
- Satu token tidak boleh punya lebih dari satu window aktif bersamaan

Kondisi gagal:

- Event dengan token_mint tidak valid: skip, log warning
- window_minutes = 0: gunakan default 5 menit, emit config warning

Integrasi:

- Input: F-02 (filtered events)
- Output: trigger_event ke F-04 (Feature Extraction)
- Konfigurasi: F-18 (trigger_engine.window_minutes, trigger_engine.mode)
- Cooldown: F-14 (dedup & cooldown)
- Monitoring: F-07 (dashboard menampilkan status window aktif)

Pastikan implementasi: Clean Architecture, SOLID Principle, Modular, Mudah dikembangkan, Mudah diuji, Production Ready

Sebelum membuat kode, lakukan analisis kebutuhan terlebih dahulu dan identifikasi seluruh komponen yang diperlukan.

FITUR 04

FEATURE EXTRACTION PIPELINE

Mengumpulkan 12+ fitur on-chain, historis, dan market context untuk XGBoost

F-04 · FEATURE EXTRACTION PIPELINE

Pipeline Ekstraksi Fitur ML

Tujuan Fitur

Saat trigger fire dari F-03, sistem harus mengumpulkan semua informasi yang diperlukan XGBoost untuk membuat keputusan. Ada tiga kelompok fitur: on-chain signals (data dari blockchain saat ini), historical wallet (rekam jejak wallet target), dan market context (kondisi pasar saat ini). Semua fitur ini dikumpulkan, dibersihkan, dan dibentuk menjadi satu feature vector.

12 Fitur yang Diekstrak

Kelompok	Fitur	Sumber Data
On-chain	position_size_usd	Event data dari trigger
On-chain	token_age_minutes	Token first seen timestamp dari RPC
On-chain	liquidity_pool_depth	DexScreener API
On-chain	slippage_actual	pump.fun quote API
On-chain	cluster_score	Jumlah correlated wallets (F-12)
Historical	win_rate_30d	Query closed_trades database
Historical	avg_holding_time_minutes	Query closed_trades database
Historical	typical_trade_size_usd	Query closed_trades database
Historical	past_exit_pattern_same_token	Query closed_trades untuk token ini
Market	sol_usd_momentum	Price feed API
Market	token_volume_liquidity_ratio	DexScreener API
Market	hour_of_day_utc	Sistem waktu lokal

Alur Bisnis

- 1

Terima Trigger Event

Menerima trigger_event dari F-03 dengan token_mint dan metadata event.
- 2

Ekstrak On-chain Features

Ambil token_age dari RPC, liquidity_pool_depth & volume_ratio dari DexScreener, slippage dari pump.fun quote.
- 3

Ekstrak Historical Features

Query closed_trades untuk wallet source: win rate 30 hari, avg holding time, typical size, past token exits.
- 4

Ekstrak Market Context

Ambil SOL/USD momentum dari price feed, hour of day dari system clock.

-
- 5 Validasi & Impute**
Cek setiap fitur: jika ada yang gagal diambil (API timeout), gunakan nilai default/impute dari data historis. Log warning.
-
- 6 Buat Feature Vector**
Gabungkan semua fitur menjadi dictionary/object yang siap diumpankan ke XGBoost (F-05).
-
- 7 Kirim ke Inference**
Teruskan feature vector beserta trigger metadata ke F-05.
-

PROMPT AGENTIC AI — F-04

Anda adalah Senior Software Engineer. Bangun fitur FEATURE EXTRACTION PIPELINE.

Tujuan fitur:

Mengumpulkan, membersihkan, dan menggabungkan 12+ fitur dari tiga sumber (on-chain, historical database, market context) menjadi satu feature vector yang siap digunakan oleh XGBoost Inference Engine (F-05).

Fungsi utama:

- Menerima trigger_event dari F-03
- Ekstrak fitur on-chain: position_size_usd, token_age_minutes, liquidity_pool_depth, slippage_actual, cluster_score
- Ekstrak fitur historis dari closed_trades database: win_rate_30d, avg_holding_time, typical_trade_size, past_exit_pattern
- Ekstrak market context: sol_usd_momentum, token_volume_liquidity_ratio, hour_of_day_utc
- Handle missing data: jika API gagal, gunakan nilai default/mean dari data historis, log warning
- Normalisasi/scaling fitur sesuai kebutuhan model
- Output: FeatureVector object dengan semua fitur + metadata trigger

Alur bisnis:

1. Terima trigger_event (token_mint, wallet_source, event_timestamp, raw_events)
2. Paralel fetch (async): on-chain data dari RPC, DexScreener, pump.fun quote
3. Query sequential: historical features dari SQLite (closed_trades WHERE wallet_source=? AND timestamp > now-30d)
4. Compute market context dari price feed + system clock
5. Validasi kelengkapan: jika fitur kritis (liquidity_pool_depth) tidak tersedia → log warning, gunakan imputed value
6. Assemble FeatureVector: dict dengan semua 12+ field + trigger_metadata
7. Enqueue ke F-05

Validasi:

- Token age tidak boleh negatif
- Win rate harus 0.0-1.0
- Jika semua historical features kosong (belum ada closed_trades): gunakan prior defaults yang dikonfigurasi
- Timeout untuk setiap API call: 5 detik, retry sekali

Kondisi gagal:

- DexScreener timeout: gunakan cached value jika ada (TTL 60 detik), otherwise impute dengan median dari dataset
- Closed trades kosong (sistem baru): return feature vector dengan historical features = prior defaults
- pump.fun quote gagal: set slippage_actual = None, XGBoost akan handle dengan imputation

Integrasi:

- Input: trigger_event dari F-03
- Fetch data: DexScreener API (A2), pump.fun API (A2), SQLite closed_trades (A1)
- Output: FeatureVector ke F-05 (XGBoost Inference)
- Caching: in-memory cache untuk DexScreener per token (TTL 60 detik, per A2)

Pastikan implementasi: Clean Architecture, SOLID Principle, Modular, Mudah dikembangkan, Mudah diuji, Production Ready

Sebelum membuat kode, lakukan analisis kebutuhan terlebih dahulu dan identifikasi seluruh komponen yang diperlukan.

FITUR 05

XGBOOST INFERENCE ENGINE

Model multi-class classifier: BUY / SELL / HOLD dengan confidence score

F-05 · XGBOOST INFERENCE ENGINE

Mesin Inferensi XGBoost

Tujuan Fitur

Setelah fitur dikumpulkan oleh F-04, XGBoost Classifier melakukan prediksi: arah trade (BUY/SELL/HOLD), confidence score (0.0-1.0), dan estimasi target harga. Model yang digunakan adalah versi aktif dari model_registry (is_active=TRUE). Hasil prediksi kemudian diteruskan ke Token Safety Check (F-06) untuk validasi tambahan sebelum menjadi alert.

Output Model

Tiga Output XGBoost

- Direction: BUY / SELL / HOLD — kelas prediksi dengan probabilitas tertinggi.
- Confidence Score: 0.0–1.0 — probabilitas kelas terpilih. Hanya confidence ≥ 0.75 yang lanjut ke alert.
- Target Price Estimate: $\pm\%$ dari harga saat ini — estimasi sederhana berdasarkan historical holding pattern.

Konfigurasi Model

XGBoost Classifier: n_estimators=300, max_depth=6, learning_rate=0.05, tree_method=hist. Dipilih karena dataset kecil (50–200 sample) — lebih stabil dari deep learning pada data sekecil ini. feature_importances_ memudahkan debugging saat model salah prediksi.

Alur Bisnis

- 1

Terima Feature Vector
Menerima FeatureVector dari F-04 beserta trigger metadata.
- 2

Load Model Aktif
Baca model_version yang is_active=TRUE dari model_registry. Load model artifact dari disk/memory.
- 3

Jalankan Inferensi
Panggil model.predict_proba(feature_vector). Ambil kelas dengan probabilitas tertinggi.
- 4

Hitung Confidence
Confidence = max(predict_proba). Tentukan direction = argmax(predict_proba).
- 5

Simpan Log Prediksi
Simpan semua prediksi ke inference_log (termasuk yang confidence < threshold) untuk training dataset.
- 6

Teruskan ke Safety Check
Kirim prediksi (direction, confidence, target_price, feature_vector, trigger_metadata) ke F-06.

PROMPT AGENTIC AI — F-05

Anda adalah Senior Software Engineer. Bangun fitur XGBOOST INFERENCE ENGINE.

Tujuan fitur:

Menjalankan prediksi menggunakan XGBoost Classifier yang sudah dilatih untuk menghasilkan direction

(BUY/SELL/HOLD), confidence score (0.0-1.0), dan target price estimate dari feature vector yang diterima dari F-04.

Fungsi utama:

- Load model aktif dari model_registry (is_active=TRUE) saat startup dan cache di memory
- Menerima FeatureVector dari F-04
- Menjalankan model.predict_proba(X) → direction + confidence
- Kalkulasi target price estimate (sederhana: $\text{entry_price} * (1 + \text{avg_r_multiple_win} * \text{risk_pct})$)
- Log SEMUA prediksi ke inference_log (termasuk yang di bawah confidence threshold)
- Meneruskan hasil ke F-06 (Token Safety Check)
- Hot-reload model saat model_registry berubah (setelah retrain F-10) tanpa restart proses

Alur bisnis:

1. Startup: query model_registry WHERE is_active=TRUE, load model artifact
2. Terima FeatureVector dari F-04
3. Transform FeatureVector ke numpy array sesuai feature order yang diharapkan model
4. Call model.predict_proba(X) → class probabilities
5. direction = kelas dengan prob tertinggi; confidence = max(proba)
6. Hitung target_price_estimate
7. Log ke inference_log: token, direction, confidence, model_version, timestamp, feature_values
8. Emit PredictionResult ke F-06

Validasi:

- Feature order harus konsisten antara training dan inference (simpan feature_names saat training)
- Model harus ada dan ter-load sebelum menerima request — jika tidak ada, reject dengan error
- Confidence harus 0.0-1.0

Kondisi gagal:

- Model tidak ada di model_registry: gunakan model bootstrap (F-11) sebagai fallback
- FeatureVector memiliki null values: impute dengan training set median sebelum inference
- Inference error (rare): log error, emit 'inference_failed' event, jangan crash sistem

Integrasi:

- Input: FeatureVector dari F-04
- Model: load dari model_registry (F-10 yang mengelola model aktif)
- Output: PredictionResult ke F-06
- Log: inference_log di SQLite untuk audit dan dataset training

Pastikan implementasi: Clean Architecture, SOLID Principle, Modular, Mudah dikembangkan, Mudah diuji, Production Ready

Sebelum membuat kode, lakukan analisis kebutuhan terlebih dahulu dan identifikasi seluruh komponen yang diperlukan.

FITUR 06

TOKEN SAFETY CHECK GATE

Validasi keamanan token: liquidity lock, kontrak verified, holder distribution

F-06 · TOKEN SAFETY CHECK GATE

Gerbang Keamanan Token

Tujuan Fitur

Mengapa Kritis

- Confidence tinggi dari XGBoost TIDAK menjamin token aman. Model hanya belajar dari pola historis, bukan dari risiko kontrak.
- Token honeypot atau rug pull bisa memiliki pola on-chain yang mirip dengan token legitimate di awal.
- Safety check ini adalah layer independen kedua — sebuah sinyal harus lolos DUA filter independen sebelum jadi alert.
- Jika API safety gagal/timeout: FAIL CLOSED — alert diblokir, bukan dilanjutkan. Prinsip: lebih baik miss satu trade daripada masuk honeypot.

4 Kriteria Safety Check

Kriteria	Cek	Fail Behavior
Liquidity Locked	Apakah LP tokens terkunci via locker yang dikenal?	BLOCK — token tanpa locked LP berisiko tinggi rug
Contract Verified	Apakah source code kontrak diverifikasi di block explorer?	BLOCK — kontrak tidak terverifikasi tidak bisa diaudit
Holder Distribution	Apakah top 10 holder menguasai < X% supply?	BLOCK — konsentrasi tinggi = risiko dump besar
Mint Authority	Apakah mint authority sudah di-revoke/disable?	BLOCK — jika masih aktif, developer bisa mint token baru k

Alur Bisnis

- 1

Terima Prediksi

Menerima PredictionResult dari F-05: direction, confidence, token_mint, feature_vector.
- 2

Cek Cache

Periksa apakah safety data untuk token ini sudah ada di cache (TTL 60 detik). Jika ada, gunakan cached result.
- 3

Fetch Safety Data

Call DexScreener/block explorer API untuk token: liquidity_lock_status, contract_verified, holder_distribution, mint_authority.
- 4

Evaluasi Kriteria

Jalankan 4 kriteria secara berurutan. Fail-fast: berhenti di kriteria pertama yang gagal.
- 5

Cek Confidence

Jika safety LOLOS, cek apakah confidence >= decision_gate.confidence_threshold (default 0.75).
- 6

Hasil Akhir

Jika lolos safety DAN confidence cukup: teruskan ke F-07 (Dashboard) untuk alert. Jika gagal: log sebagai 'blocked', tetap masuk dataset training.

PROMPT AGENTIC AI — F-06

Anda adalah Senior Software Engineer. Bangun fitur TOKEN SAFETY CHECK GATE.

Tujuan fitur:

Layer keamanan independen yang memvalidasi token sebelum alert dikirim ke dashboard. Menggunakan data publik dari DexScreener dan block explorer untuk memastikan token tidak memiliki karakteristik honeypot/rug pull.

Fungsi utama:

- Menerima PredictionResult dari F-05
- Fetch token safety data dari DexScreener API dan/atau block explorer (per A2)
- Evaluasi 4 kriteria: liquidity_locked, contract_verified, holder_distribution_safe, mint_authority_revoked
- Implementasi in-memory cache per token (TTL 60 detik) untuk hindari API calls berulang
- Fail-closed behavior: jika API timeout/error → block alert, log 'safety_check_failed'
- Hasil: SafetyCheckResult (passed/failed + reason) diteruskan ke F-07 atau logged ke dataset

Alur bisnis:

1. Terima PredictionResult (direction, confidence, token_mint, feature_vector, trigger_metadata)
2. Check cache: jika token_mint ada di cache dan belum expired → gunakan cached result
3. Async fetch dari DexScreener: GET /token/{token_mint} → parse liquidity_locked, contract_verified, top_holders, mint_authority
4. Jika fetch gagal (timeout 5 detik): fail-closed → log 'safety_api_failed', emit 'blocked' signal ke F-07, masuk dataset
5. Evaluasi criteria (fail-fast): lp_locked? → contract_verified? → holder_concentration < threshold? → mint_revoked?
6. Update cache dengan result
7. Jika LOLOS safety: cek confidence >= confidence_threshold (dari F-18)
8. Jika lolos semua: emit AlertSignal ke F-07; jika tidak: emit LogOnlySignal ke dataset

Validasi:

- holder_distribution threshold dari config.yaml (default: top 10 holder < 20%)
- confidence_threshold dari config.yaml decision_gate.confidence_threshold
- Cache tidak boleh melebihi N entries (LRU eviction untuk memory bound)

Kondisi gagal:

- API down: fail-closed (blokir alert), jangan fail-open
- Holder data tidak tersedia di API: conservative assumption = FAIL safety check
- Token tidak ditemukan di DexScreener: FAIL safety check

Integrasi:

- Input: PredictionResult dari F-05
- API: DexScreener/block explorer (A2)
- Output (lolos): AlertSignal ke F-07 (Dashboard)
- Output (gagal): LogOnlySignal → dataset training (tetap dicatat untuk F-10)
- Konfigurasi: F-18 untuk threshold holder dan confidence

Pastikan implementasi: Clean Architecture, SOLID Principle, Modular, Mudah dikembangkan, Mudah diuji, Production Ready

Sebelum membuat kode, lakukan analisis kebutuhan terlebih dahulu dan identifikasi seluruh komponen yang diperlukan.

FITUR 07

WEB DASHBOARD ALERT ENGINE

Real-time browser dashboard via WebSocket push — wallet monitor, signal feed, trade log

F-07 · WEB DASHBOARD ALERT ENGINE

Dashboard Alert Real-Time

Tujuan Fitur

Dashboard adalah antarmuka utama bagi pengguna untuk memantau seluruh aktivitas sistem. Data dikirim via WebSocket push (bukan polling) sehingga sinyal baru muncul seketika. Dashboard menampilkan status wallet, live signal feed, confidence history, trade log, dan statistik harian. Ini adalah satu-satunya interface antara sistem otomatis dan pengguna.

Komponen Dashboard

Panel	Konten	Update Frequency
Wallet Monitor	Status Whale A & B (active/idle), trigger rate per jam, last trigger time	Real-time via WS
Live Signal Feed	Daftar sinyal terbaru: token, direction, confidence, timestamp	Push saat ada sinyal baru
Confidence Score	Score sinyal terakhir (besar), threshold reference line	Real-time
Confidence History (24h)	Bar chart confidence semua sinyal dalam 24 jam dengan threshold	Update per sinyal
Trade Log	Ringkasan posisi: token, direction, PnL, status (open/closed)	Real-time
Win Rate	Win rate model aktif berdasarkan closed_trades 30 hari	Update setelah posisi closed
Triggers Today	Jumlah trigger yang terjadi hari ini	Real-time
Alerts Fired	Jumlah alert yang dikirim (lolos safety + confidence) vs total trigger	Real-time
System Status	Status RPC connection, model version aktif, next retrain time	Polling 30 detik

Alur Bisnis

- 1

Sinyal Masuk

AlertSignal masuk dari F-06. Dashboard backend menerima dan memformat data untuk WebSocket broadcast.
- 2

WebSocket Broadcast

Server mengirim event ke semua browser client yang terhubung: signal_new, trade_opened, trade_closed, system_status.
- 3

Browser Update

JavaScript client menerima event dan update UI panel yang relevan tanpa refresh halaman.
- 4

Notifikasi Alert

Jika direction=BUY dan confidence >= threshold: tampilkan notifikasi visual + suara (opsional).
- 5

Interaksi Manual

Pengguna dapat melihat detail sinyal, approve/reject dynamic wallet discovery (F-12), dan akses trade log.
- 6

Polling System Status

Setiap 30 detik, client poll endpoint untuk status RPC, model version, uptime.

Skenario Penggunaan

Skenario 1 — Sinyal BUY Masuk

Tujuan: Pengguna melihat sinyal BUY baru secara real-time.

Langkah: Whale A & B swap ke token ABC → sistem proses → confidence 89% → safety lolos → AlertSignal dikirim ke dashboard WS.

Hasil: Panel Live Signal Feed update: 'BUY · ABC · 89% conf · 14:32 UTC'. Notifikasi visual muncul.

Skenario 2 — Posisi Ditutup oleh Kill-Switch

Tujuan: Pengguna melihat posisi ditutup otomatis karena rug terdeteksi.

Langkah: Kill-switch (F-09) deteksi LP removal pada token XYZ → exit market order → posisi closed.

Hasil: Panel Trade Log update: 'XYZ · CLOSED · exit_reason: kill_switch_lp · PnL: -12%'. Dashboard menampilkan notifikasi merah.

PROMPT AGENTIC AI — F-07

Anda adalah Senior Software Engineer. Bangun fitur WEB DASHBOARD ALERT ENGINE.

Tujuan fitur:

Menyediakan antarmuka web browser real-time yang menampilkan seluruh aktivitas sistem — wallet monitoring, sinyal trading, trade log, dan statistik — menggunakan WebSocket push tanpa polling.

Fungsi utama:

Backend:

- WebSocket server untuk broadcast event ke connected clients
- REST API endpoint untuk: signal history, trade log, system status, approval dynamic wallet discovery
- Event types: signal_new, trade_opened, trade_closed, position_updated, system_status, rpc_status_change
- Query layer untuk: recent signals (24h), open positions, closed trades (30d), win rate, trigger count

Frontend (browser):

- WebSocket client dengan auto-reconnect
- Panel: Wallet Monitor, Live Signal Feed, Confidence Chart (24h), Trade Log, Win Rate, System Status
- Confidence history chart: bar chart dengan threshold reference line
- Notifikasi visual untuk sinyal BUY dengan confidence >= threshold
- Dynamic Wallet Approval UI: daftar kandidat wallet auto-discovered (F-12) dengan tombol approve/reject
- No mobile app — browser-only, responsive untuk desktop

Alur bisnis:

1. Client buka browser, load dashboard, establish WebSocket connection
2. Server kirim initial_state: posisi terbuka, 24h signal history, system status
3. Real-time: setiap event baru → server broadcast ke semua connected clients

4. Client update panel yang relevan tanpa full reload
5. Alert: jika signal_new dengan direction=BUY dan confidence $\geq$ threshold → notifikasi visual
6. Trade log: update saat trade_opened, position_updated, trade_closed events masuk

Validasi:

- Confidence chart harus menampilkan threshold line dari config (bukan hardcode 75%)
- WebSocket reconnect otomatis jika koneksi putus (dengan exponential backoff)
- Data sensitif (wallet address) ditampilkan partial (first 6 + ... + last 4 chars)

Hak akses:

- Dashboard hanya accessible dari localhost (tidak exposed ke internet tanpa auth)
- Dynamic wallet approval: hanya pengguna yang dapat approve/reject (catat approval_timestamp + action)

Kondisi gagal:

- WebSocket server down: client tampilkan 'Disconnected' badge, retry reconnect otomatis
- Database query slow: tampilkan loading state, timeout 3 detik, tampilkan cached data jika ada

Integrasi:

- Input: AlertSignal dari F-06, position updates dari F-08, F-09, system status dari seluruh komponen
- Output: approval events dari user ke F-12 (dynamic wallet discovery)
- Config: F-18 untuk confidence threshold display

Pastikan implementasi: Clean Architecture, SOLID Principle, Modular, Mudah dikembangkan, Mudah diuji, Production Ready

Sebelum membuat kode, lakukan analisis kebutuhan terlebih dahulu dan identifikasi seluruh komponen yang diperlukan.

FITUR 08

AUTO TRADE EXECUTION

Auto-entry via pump.fun API — size dari 1% risk, konfirmasi, state persistence

F-08 · AUTO TRADE EXECUTION

Eksekusi Trade Otomatis

Tujuan Fitur

Konteks Kritis

- Fitur ini berinteraksi langsung dengan uang riil. Setiap bug di sini berarti kerugian finansial nyata.
- Auto-entry hanya dijalankan jika: AlertSignal masuk (confidence >= 75% DAN safety lolos) DAN Position Correlation Cap (F-16) tidak terlampaui.
- Position size selalu dihitung dari 1% risk — BUKAN persentase harga token. Stop loss otomatis menyesuaikan size.
- Private key harus tersimpan terenkripsi lokal. TIDAK PERNAH plaintext. TIDAK PERNAH keluar dari mesin lokal.

Mekanisme Position Sizing

Risk per trade = 1% dari equity total. $\text{Position size} = (\text{equity} * 0.01) / \text{distance_to_SL_in_pct}$. Stop loss awal = -1R dari entry price. Ini membuat R-multiple konsisten secara matematis di seluruh sistem. Contoh: equity \$1000, SL 10% dari entry → $\text{position size} = (\$1000 * 0.01) / 0.10 = \$100$.

Alur Bisnis

- 1

Terima AlertSignal

Menerima AlertSignal dari F-06 (sudah lolos safety + confidence check).
- 2

Cek Correlation Cap

Cek F-16: apakah jumlah open_positions sudah mencapai N maksimal? Jika ya → log sinyal, skip eksekusi.
- 3

Hitung Position Size

Kalkulasi size berdasarkan equity saat ini * risk_pct (1%) / jarak SL dari config.
- 4

Get Quote

Call pump.fun quote API untuk mendapatkan harga saat ini dan estimasi slippage.
- 5

Place Entry Order

Sign dan submit market order ke pump.fun API dengan position size yang dihitung.
- 6

Konfirmasi Transaksi

Poll transaction status sampai confirmed (atau timeout). Jika gagal, retry max 2x.
- 7

Simpan State Posisi

Tulis ke open_positions di SQLite: token, entry_price, size, SL_level, entry_timestamp, model_version.
- 8

Aktivasi Proteksi

Trigger F-09 (Three-Layer Protection) untuk posisi ini.
- 9

Update Dashboard

Kirim trade_opened event ke F-07.

PROMPT AGENTIC AI — F-08

Anda adalah Senior Software Engineer. Bangun fitur AUTO TRADE EXECUTION.

Tujuan fitur:

Mengeksekusi entry trade otomatis via pump.fun API berdasarkan AlertSignal yang sudah lolos semua validasi.

Position size dihitung dari fixed 1% risk. State posisi disimpan ke database segera setelah order dikonfirmasi.

Fungsi utama:

- Menerima AlertSignal dari F-06
- Cek Position Correlation Cap (F-16) sebelum eksekusi
- Kalkulasi `position_size_usd`: $(current_equity * risk_pct) / sl_distance_pct$
- Fetch real-time quote dari pump.fun untuk harga aktual dan slippage estimate
- Sign dan submit market entry order ke pump.fun API menggunakan wallet keypair lokal terenkripsi
- Poll konfirmasi transaksi (max 10 detik, retry 2x jika gagal)
- Simpan open position ke SQLite SEGERA setelah order confirmed (bukan setelah semua proses selesai)
- Trigger F-09 untuk mengaktifkan 3-layer proteksi
- Emit `trade_opened` event ke F-07

Validasi:

- Hanya eksekusi jika `is_auto_trade_enabled=TRUE` di config
- Position size tidak boleh melebihi `max_position_size_usd` di config (safety guard)
- Slippage estimate tidak boleh melebihi `slippage_tolerance` di config
- Equity harus di-fetch fresh sebelum setiap entry (bukan cached)

Hak akses:

- Private key wallet hanya accessible oleh komponen ini melalui secure key manager (terenkripsi, AES-256)
- Tidak ada akses eksternal ke fungsi eksekusi

Kondisi gagal:

- Order gagal setelah 2 retry: log sebagai 'missed_entry', jangan masuk `closed_trades` (posisi tidak pernah terbuka)
- Quote timeout: batalkan eksekusi, log 'quote_failed'
- Slippage terlalu tinggi: batalkan eksekusi, log 'slippage_too_high'
- Database write gagal setelah order confirmed: CRITICAL — retry database write dengan exponential backoff, jangan lanjut ke F-09 sampai berhasil

Audit trail:

- Setiap eksekusi dicatat: timestamp, token, direction, size, entry_price, confidence, model_version, correlation_cap_status
- Setiap kegagalan dicatat dengan alasan dan context lengkap

Integrasi:

- Input: AlertSignal dari F-06
- Pre-check: F-16 (Correlation Cap)
- API: pump.fun (A2) untuk quote dan order

- State: SQLite open_positions (A6)
- Aktivasi: F-09 (Three-Layer Protection)
- Monitoring: F-07 (Dashboard)

Pastikan implementasi: Clean Architecture, SOLID Principle, Modular, Mudah dikembangkan, Mudah diuji, Production Ready

Sebelum membuat kode, lakukan analisis kebutuhan terlebih dahulu dan identifikasi seluruh komponen yang diperlukan.

FITUR 09

THREE-LAYER POSITION PROTECTION

SL, Trailing TP bertingkat, Kill-Switch on-chain — berjalan paralel

F-09 · THREE-LAYER POSITION PROTECTION

Sistem Proteksi Posisi Tiga Lapis

Tujuan Fitur

Begitu posisi terbuka, tiga mekanisme proteksi berjalan paralel dan independen. Lapis 1 (Price SL) adalah baseline yang selalu aktif. Lapis 2 (Trailing TP) mengikuti profit dengan trail yang makin ketat seiring R naik. Lapis 3 (Kill-Switch) beroperasi independen dari harga — mendeteksi event on-chain berbahaya dan memiliki PRIORITAS TERTINGGI yang override lapis lain.

Struktur Tiga Lapis

Lapis	Mekanisme	Trigger Condition	Prioritas
Lapis 1	Price-Based SL (Fixed -1R)	Harga <= entry_price * (1 - sl_pct)	Normal — selalu aktif
Lapis 2	Staged Trailing TP (Makin ketat per R tier)	+1R-2R: SL tetap +2R-5R: trail 25% +5R-10R: trail 15% >+10R: trail 10%	Normal — aktif saat >+2R
Lapis 3	On-Chain Kill-Switch (Independen harga)	LP removal, Dev wallet dump besar, Slippage spike, Holder shift	TERTINGGI — override semua

Alur Bisnis

- 1

Aktivasi Tiga Lapis

F-08 menginisialisasi: Lapis 1 (async task), Lapis 2 (async task), Lapis 3 (thread terpisah priority) untuk posisi ini.
- 2

Lapis 1 — Price Check

Setiap kali harga update masuk: bandingkan current_price dengan SL level. Jika <= SL → trigger exit market order.
- 3

Lapis 2 — Trail Update

Setiap harga update: hitung R saat ini = (current_price - entry) / (entry - sl_price). Update trailing SL sesuai tier. Jika harga turun ke trailing SL → trigger exit.
- 4

Lapis 3 — On-Chain Monitor

Thread terpisah subscribe ke RPC events untuk token posisi: LP removal, dev wallet transactions, Jupiter/pump.fun slippage quotes, holder concentration changes.
- 5

Kill-Switch Trigger

Jika salah satu event Lapis 3 terdeteksi → LANGSUNG kirim market exit order, TANPA menunggu evaluasi Lapis 1/2.
- 6

Exit Dikonfirmasi

Setelah exit order confirmed: tutup semua lapis proteksi, pindahkan dari open_positions ke closed_trades, set exit_reason, hitung r_multiple.
- 7

Feed ke Training

Closed trade masuk ke dataset training untuk retrain berikutnya (F-10).

PROMPT AGENTIC AI — F-09

Anda adalah Senior Software Engineer. Bangun fitur THREE-LAYER POSITION PROTECTION.

Tujuan fitur:

Menjalankan tiga mekanisme proteksi posisi secara paralel dan independen begitu sebuah posisi dibuka oleh F-08.

Lapis 3 (Kill-Switch on-chain) memiliki prioritas tertinggi dan dapat override Lapis 1 & 2.

Fungsi utama:

LAPIS 1 — Price-Based SL (async task):

- Monitor `current_price` via price feed event
- Jika $\text{current_price} \leq \text{entry_price} * (1 - \text{sl_distance_pct}) \rightarrow$ trigger exit order
- `sl_distance_pct` dihitung saat entry: $\text{risk_pct} / \text{position_size_pct}$

LAPIS 2 — Staged Trailing TP (async task, shared event loop dengan Lapis 1):

- Hitung $R_{\text{current}} = (\text{current_price} - \text{entry_price}) / (\text{entry_price} - \text{initial_sl_price})$
- Evaluasi tier berdasarkan config `trailing_tp.tiers`
- Update `trailing_sl = peak_price * (1 - trail_pct)` untuk tier aktif
- Jika $\text{current_price} \leq \text{trailing_sl} \rightarrow$ trigger exit order

LAPIS 3 — On-Chain Kill-Switch (thread terpisah, event-driven via WebSocket RPC):

- Subscribe ke token address untuk: LP events, holder changes
- Poll Jupiter/pump.fun quote setiap N detik untuk slippage check
- Monitor dev wallet transactions (dari watchlist atau known dev addresses)
- Trigger conditions: LP removal/burn, $\text{dev_wallet_sell} > \text{threshold_pct}$, $\text{slippage} > \text{spike_threshold}$, $\text{holder_concentration_shift} > \text{threshold}$
- Saat trigger: kirim sinyal ke `shared kill_switch_event` (threading.Event) yang dicek oleh Lapis 1&2 SEBELUM setiap evaluasi
- Langsung execute market exit order tanpa menunggu

Exit Order Execution (shared across layers):

- Satu kali eksekusi: gunakan atomic flag agar tidak double-exit
- Place market order ke pump.fun API
- Jika gagal: retry agresif dengan slippage tolerance dinaikkan bertahap (0.5% $\rightarrow$ 1% $\rightarrow$ 2% $\rightarrow$ 5%)
- Konfirmasi $\rightarrow$ close all layers $\rightarrow$ move `open_positions` ke `closed_trades` $\rightarrow$ set `exit_reason`

Validasi:

- Trailing SL tidak boleh lebih rendah dari initial SL (hanya bergerak ke atas/naik)
- Kill-switch threshold dari config.yaml (`kill_switch` section)
- Double-exit prevention: atomic lock sebelum execute exit order

Kondisi gagal:

- Exit order gagal saat kill-switch: retry agresif tanpa jeda, slippage tolerance maksimal
- Price feed terputus: gunakan L3 kill-switch sebagai primary monitor, L1&2 masuk degraded mode

- Thread Lapis 3 crash: watchdog restart thread, emit critical alert ke dashboard

Audit:

- Log setiap evaluasi Lapis 1&2 setiap N detik dengan R_current, trailing_sl_current
- Log setiap event yang dideteksi Lapis 3 dengan detail
- exit_reason: SL / trailing_tp / kill_switch_lp / kill_switch_dev_dump / kill_switch_slippage

Integrasi:

- Aktivasi oleh: F-08 (setelah open_position tersimpan)
- Exit execution: pump.fun API (A2)
- State: SQLite open_positions → closed_trades (A1, A6)
- Dashboard: F-07 untuk position updates
- Training: closed trade otomatis tersedia untuk F-10

Pastikan implementasi: Clean Architecture, SOLID Principle, Modular, Mudah dikembangkan, Mudah diuji, Production Ready

Sebelum membuat kode, lakukan analisis kebutuhan terlebih dahulu dan identifikasi seluruh komponen yang diperlukan.

FITUR 10

24H RETRAIN & ROLLBACK SCHEDULER

Cron job 02:00 UTC — retrain XGBoost dari closed trades, rollback guard otomatis

F-10 · 24H RETRAIN & ROLLBACK SCHEDULER

Penjadwal Retrain Harian dengan Rollback Guard

Tujuan Fitur

Model XGBoost diperbarui setiap 24 jam menggunakan data closed trades terbaru. Ini memastikan model terus beradaptasi dengan perubahan perilaku whale target. Jika model baru justru lebih buruk dari model sebelumnya, rollback otomatis dilakukan. Retrain berjalan sebagai proses terpisah agar tidak mengganggu monitoring real-time.

Kriteria Retrain

Kapan Retrain Dijalankan

- Jadwal normal: Cron 02:00 UTC setiap hari (volume pasar rendah, minimal gangguan).
- Catch-up: Saat startup, cek timestamp retrain terakhir di model_registry. Jika > 24 jam, langsung retrain.
- Syarat minimum data: 100-150 closed trades, ATAU 50 trades dengan minimal 15-20 berlabel BUY_BENAR.
- Dataset: rolling window 30 hari dari closed_trades. Hanya data real (paper + live), BUKAN mock.

Rollback Guard

Kondisi Rollback Otomatis

- Validation accuracy turun > 5% dibanding model sebelumnya.
- Expectancy(R) berubah dari positif ke negatif: $Expectancy = (WR \times avg_R_win) - ((1-WR) \times 1R)$.
- Jika rollback: set is_active kembali ke model sebelumnya, tandai model baru sebagai rolled_back=TRUE.
- Notifikasi rollback dikirim ke dashboard sebagai system alert.

Alur Bisnis

- 1

Jadwal Trigger
Cron job 02:00 UTC aktif, atau catch-up saat startup jika retrain terlewat.
- 2

Validasi Data
Query closed_trades rolling 30 hari. Cek apakah memenuhi syarat minimum sample. Jika tidak cukup data → skip, log, tunggu besok.
- 3

Labeling Dataset
Hitung $r_multiple = pnl_pct_actual / risk_pct$. Label: BUY_BENAR ($\geq +3.0R$), SALAH ($\leq -1.0R$), HOLD (di antaranya).
- 4

Training
Jalankan XGBoost training dengan konfigurasi dari config.yaml (n_estimators, max_depth, dll) sebagai proses terpisah. Target < 2 menit.
- 5

Evaluasi
Hitung validation_accuracy dan expectancy_r pada validation set (holdout 20%).

6 Rollback Guard

Bandingkan dengan model sebelumnya. Jika ada drop signifikan: rollback ke model lama, mark model baru rolled_back=TRUE.

7 Aktivasi Model Baru

Jika lolos: set is_active=TRUE untuk model baru, set FALSE untuk yang lama. Update model_registry.

8 Hot Reload

Kirim sinyal ke F-05 untuk reload model aktif tanpa restart proses.

PROMPT AGENTIC AI — F-10

Anda adalah Senior Software Engineer. Bangun fitur 24H RETRAIN & ROLLBACK SCHEDULER.

Tujuan fitur:

Menjalankan retraining XGBoost secara terjadwal (02:00 UTC) menggunakan data closed trades terbaru, dengan validasi kualitas model dan rollback otomatis jika model baru lebih buruk.

Fungsi utama:

- Cron scheduler (02:00 UTC) atau catch-up saat startup (jika last_retrain > 24h)
- Query closed_trades: rolling 30 hari, WHERE exit_reason NOT LIKE 'kill_switch%' untuk evaluasi sinyal murni
- Validasi minimum sample: 100+ trades OR (50+ trades AND BUY_BENAR count >= 15)
- Labeling: hitung r_multiple, assign label berdasarkan thresholds dari config
- Handle class imbalance: scale_pos_weight = count(SALAH) / count(BUY_BENAR)
- Training XGBoost sebagai subprocess/process terpisah (target < 2 menit)
- Evaluasi: validation_accuracy + expectancy_r pada holdout 20%
- Rollback guard: bandingkan dengan model is_active=TRUE sebelumnya
- Jika lolos: update model_registry (set old is_active=FALSE, new is_active=TRUE), emit model_updated event ke F-05
- Jika rollback: set rolled_back=TRUE pada model baru, keep model lama aktif, emit rollback_alert ke F-07

Alur bisnis:

1. Trigger (cron atau catch-up)
2. Query dan validasi dataset
3. Jika tidak cukup data: log 'retrain_skipped_insufficient_data', schedule ulang untuk besok
4. Build feature matrix X, label vector y dari closed_trades
5. Split train/val (80/20 stratified)
6. Train XGBoost (subprocess agar tidak block monitoring)
7. Evaluate: accuracy, expectancy_r pada val set
8. Rollback check vs current active model
9. Update model_registry dan trigger hot reload ke F-05
10. Log complete retrain event dengan: sample_count, accuracy, expectancy_r, duration, rollback_status

Validasi:

- Label thresholds dari config: labeling.buy_benar_threshold_r, labeling.salah_threshold_r
- XGBoost params dari config: n_estimators, max_depth, learning_rate
- Rollback threshold dari config: retrain.rollback_accuracy_drop_pct (default 0.05)
- Hanya satu model yang boleh is_active=TRUE pada satu waktu (database constraint)

Kondisi gagal:

- Training process crash: log error, keep model lama aktif, alert ke dashboard
- Database write gagal setelah training: retry 3x, jika gagal: keep model lama, alert critical

Audit trail:

- Setiap retrain event lengkap: timestamp_start, timestamp_end, sample_count, validation_accuracy, expectancy_r, action (activated/rolled_back/skipped)

Integrasi:

- Dataset: SQLite closed_trades (A1)
- Model storage: SQLite model_registry (A1) + model file on disk
- Hot reload: sinyal ke F-05 (XGBoost Inference)
- Alert: F-07 (Dashboard) untuk retrain complete / rollback events
- Konfigurasi: F-18 (semua parameter retrain)

Pastikan implementasi: Clean Architecture, SOLID Principle, Modular, Mudah dikembangkan, Mudah diuji, Production Ready

Sebelum membuat kode, lakukan analisis kebutuhan terlebih dahulu dan identifikasi seluruh komponen yang diperlukan.

FITUR 11

MODEL v0 BOOTSTRAP

Cold-start: latih model awal dari histori on-chain wallet sebelum 50 closed trades live

F-11 · MODEL v0 BOOTSTRAP

Bootstrap Model Awal dari Histori On-Chain

Tujuan Fitur

Masalah Cold-Start

- Tanpa fitur ini, sistem butuh 50-100 closed trades live sebelum bisa training model pertama — bisa butuh berminggu-minggu.
- Selama periode itu, sistem tidak punya model valid dan tidak bisa menghasilkan sinyal berkualitas.
- Solusi: tarik histori transaksi wallet target dari block explorer publik, rekonstruksi label dari pergerakan harga riil pasca-transaksi, dan latih model v0.
- Model bootstrap ini otomatis digantikan oleh model v1 begitu data live mencukupi (F-10).

Alur Bisnis

- 1 Trigger Bootstrap**
Saat startup, cek model_registry. Jika tidak ada model (is_active=TRUE), jalankan bootstrap.
- 2 Fetch Histori Wallet**
Query histori transaksi Whale A & B dari block explorer publik (free API) untuk N hari ke belakang.
- 3 Rekonstruksi Entry/Exit**
Identifikasi pola entry (swap in) dan exit (swap out) dari histori. Hitung holding time dan token yang diperdagangkan.
- 4 Label dari Harga Riil**
Fetch harga historis token dari DexScreener untuk setiap posisi rekonstruksi. Hitung PnL dan r_multiple dari pergerakan harga riil.
- 5 Training Model v0**
Jalankan XGBoost training dengan dataset historis yang sudah dilabel. Simpan sebagai model_version='v0'.
- 6 Aktivasi**
Set is_active=TRUE untuk v0 di model_registry. F-05 otomatis menggunakan v0 untuk inferensi.
- 7 Penggantian Otomatis**
Saat F-10 retrain pertama kali dengan data live mencukupi, v0 akan digantikan oleh v1 melalui mekanisme rollback guard normal.

PROMPT AGENTIC AI — F-11

Anda adalah Senior Software Engineer. Bangun fitur MODEL v0 BOOTSTRAP.

Tujuan fitur:

Mengatasi cold-start problem dengan melatih model XGBoost awal (v0) dari histori on-chain transaksi wallet target yang diambil dari block explorer publik, sehingga sistem punya model valid sejak hari pertama.

Fungsi utama:

- Cek model_registry saat startup: jika tidak ada is_active=TRUE → jalankan bootstrap
- Fetch transaksi historis Whale A & B dari Solana block explorer publik (free API)
- Parse transaksi untuk identifikasi: token_mint, direction (buy/sell), amount, timestamp
- Rekonstruksi posisi: match entry (swap in) dengan exit (swap out) berdasarkan token dan waktu
- Fetch harga historis token dari DexScreener untuk rekonstruksi PnL
- Labeling: hitung r_multiple dari PnL riil, assign BUY_BENAR/SALAH/HOLD
- Feature engineering: ekstrak semua fitur yang sama dengan F-04 untuk setiap historical trade
- Training XGBoost v0 dengan dataset historis
- Simpan ke model_registry sebagai model_version='v0', is_active=TRUE
- Tandai semua bootstrap trades di database sebagai is_bootstrap=TRUE

Validasi:

- Histori minimal N hari (configurable) untuk memastikan sample cukup beragam
- Jika histori tidak cukup: training dengan data yang ada, log warning tentang keterbatasan data
- Bootstrap hanya dijalankan SEKALI saat tidak ada model — tidak bisa di-trigger manual (keamanan)

Kondisi gagal:

- Block explorer API tidak tersedia: log error, sistem berjalan tanpa model v0 (fallback: rule-based saja)
- Rekonstruksi posisi gagal (data tidak konsisten): skip posisi tersebut, log, lanjut dengan yang lain
- Training gagal: log error, sistem tetap berjalan dengan mode manual (no auto-alert)

Integrasi:

- Input: block explorer API (public, free)
- Harga historis: DexScreener API
- Output: model_registry (v0, is_active=TRUE) + model file on disk
- Aktivasi: F-05 (XGBoost Inference) otomatis load v0
- Replacement: F-10 (Retrain Scheduler) akan replace v0 dengan v1 setelah data live cukup

Pastikan implementasi: Clean Architecture, SOLID Principle, Modular, Mudah dikembangkan, Mudah diuji, Production Ready

Sebelum membuat kode, lakukan analisis kebutuhan terlebih dahulu dan identifikasi seluruh komponen yang diperlukan.

FITUR 12-16

ENHANCEMENT FEATURES B1–B5

5 penguatan robustness: Dynamic Discovery, Hard Filter, Dedup, RPC Fallback, Correlation Cap

F-12 · DYNAMIC WALLET DISCOVERY (B1)

Penemuan Wallet Baru Secara Otomatis

Sistem mengidentifikasi wallet eksternal yang secara konsisten masuk ke token yang sama dengan Whale A/B dalam waktu berdekatan dan profit. Kandidat tampil di dashboard untuk approval manual — TIDAK otomatis aktif tanpa persetujuan pengguna. Ini murni rule-based, berjalan sebagai background process ringan.

PROMPT AGENTIC AI — F-12

Anda adalah Senior Software Engineer. Bangun fitur DYNAMIC WALLET DISCOVERY (Enhancement B1).

Tujuan fitur:

Secara otomatis mengidentifikasi wallet eksternal yang konsisten profit bersama Whale A/B pada token yang sama,

dan mengusulkan wallet tersebut sebagai kandidat watchlist baru untuk approval manual.

Fungsi utama:

- Background process (berjalan terpisah dari real-time pipeline, tidak menambah beban RPC)
- Setiap kali Whale A/B trading token X, record wallet lain yang masuk token X dalam window ± 10 menit
- Agregasi: jika wallet eksternal muncul $\geq N$ kali (configurable, default 3) bersama whale target
- Verifikasi profit: cek exit price vs entry price dari data on-chain publik
- Jika memenuhi kriteria: insert ke watchlist_wallets dengan source='auto_discovered', active=FALSE
- Kirim notifikasi ke F-07 (dashboard): tampilkan kandidat wallet dengan stats
- Pengguna approve/reject via dashboard (F-07)
- Jika approved: set active=TRUE, wallet mulai dimonitor oleh F-01

Validasi:

- Threshold kemunculan (N) dari config
- Wallet tidak boleh sudah ada di watchlist (dedup by wallet_address)
- Verifikasi profit minimal dari X% dari transaksi yang tercatat

Kondisi gagal:

- Data on-chain tidak cukup untuk verifikasi profit: tandai kandidat sebagai 'unverified', tetap tampilkan untuk review manual
- Block explorer rate limit: queue verification, proses dengan jeda

Integrasi:

- Data source: event stream dari F-01 (baca data yang sudah ada, tidak tambah RPC call baru)
- Output: watchlist_wallets (SQLite, A1)
- Notifikasi: F-07 (dashboard approval UI)
- Aktivasi: F-01 otomatis subscribe ke wallet baru jika approved

Pastikan implementasi: Clean Architecture, SOLID Principle, Modular, Mudah dikembangkan, Mudah diuji, Production Ready

F-13 · TOKEN AGE & LIQUIDITY HARD FILTER (B2)

Filter Keras Age & Liquiditas Token

Hard filter di Layer 1 — sebelum event masuk trigger window. Token yang terlalu baru (usia di bawah minimum) atau dengan liquidity pool terlalu dangkal langsung diabaikan dan TIDAK masuk dataset training sama sekali. Berbeda dari token dengan confidence rendah yang tetap di-log — token yang gagal hard filter benar-benar dibuang dari pipeline.

PROMPT AGENTIC AI — F-13

Anda adalah Senior Software Engineer. Bangun fitur TOKEN AGE & LIQUIDITY HARD FILTER (Enhancement B2).

Tujuan fitur:

Hard filter di Layer 1 yang membuang event dari token terlalu baru atau liquidity terlalu dangkal SEBELUM masuk trigger window — mencegah noise ekstrem mencemari training dataset.

Fungsi utama:

- Disisipkan di antara F-02 (filter relevansi) dan F-03 (trigger engine)
- Untuk setiap event lolos F-02: fetch token_age_minutes dan liquidity_pool_depth
- Jika token_age_minutes < min_token_age_minutes (config): DISCARD, tidak di-log ke training data
- Jika liquidity_pool_depth < min_liquidity_usd (config): DISCARD, tidak di-log ke training data
- Jika lolos: teruskan event ke F-03 dengan token age & liquidity sebagai metadata
- Log ke hard_filter_log untuk audit: token, reason (age/liquidity), values

Validasi:

- min_token_age_minutes dan min_liquidity_usd dari config.yaml (perlu dikalibrasi dari data live)
- Jika DexScreener API gagal untuk fetch data: conservative approach → DISCARD (fail-safe)
- Cache token metadata per token_mint (TTL 5 menit) untuk hindari API calls berulang

Kondisi gagal:

- DexScreener down: semua event discard sementara (tidak ada token yang lolos tanpa verifikasi age/liquidity)
- Token tidak ditemukan di DexScreener: DISCARD

Integrasi:

- Input: filtered events dari F-02
- Data fetch: DexScreener API (jika tidak tersedia dari F-04 cache)
- Output: events ke F-03 (Trigger Engine)
- Konfigurasi: F-18

Pastikan implementasi: Clean Architecture, SOLID Principle, Modular, Production Ready

F-14 · DEDUP & COOLDOWN PER (WALLET, TOKEN) (B3)

Pencegah Alert Ganda per Pasangan Wallet-Token

Ketika whale membangun posisi secara bertahap (beberapa transaksi ke token yang sama dalam waktu pendek), sistem berisiko menganggap ini sebagai multiple sinyal independen. Cooldown per (wallet, token) mencegah hal ini — satu pasangan whale+token hanya bisa memicu satu trigger baru selama posisi terkait masih terbuka. Reset otomatis saat posisi ditutup.

PROMPT AGENTIC AI — F-14

Anda adalah Senior Software Engineer. Bangun fitur DEDUP & COOLDOWN per (WALLET, TOKEN) (Enhancement B3).

Tujuan fitur:

Mencegah trigger ganda dari whale yang sama ke token yang sama dalam period singkat, yang bisa menyebabkan

alert fatigue dan duplikasi di dataset training.

Fungsi utama:

- Cooldown table di SQLite: key=(wallet_address, token_address), value=timestamp_trigger + status
- Dicek di F-03 (Trigger Engine) sebelum window evaluation
- Jika (wallet, token) dalam cooldown: event masih bisa memperbarui data posisi terbuka, tapi TIDAK bisa trigger baru
- Cooldown persist ke SQLite (A6 — state persistence) agar tidak hilang saat restart
- Reset cooldown otomatis saat posisi untuk token tersebut ditutup (event dari F-09)
- Durasi cooldown: per posisi aktif (bukan fixed timer) — cooldown berakhir saat posisi closed

Validasi:

- Cooldown tidak boleh memblokir monitoring posisi yang sudah ada (hanya blokir trigger BARU)
- State harus persist ke SQLite saat write dan bisa di-restore saat startup (A6)
- Sinkronisasi thread-safe untuk cooldown table (bisa diakses dari multiple async tasks)

Kondisi gagal:

- SQLite write gagal untuk cooldown: use in-memory fallback, log warning (sistem tetap jalan)
- Cooldown expired secara salah: worst case adalah satu alert duplikat — acceptable, tidak kritis

Integrasi:

- Dicek oleh: F-03 (Trigger Engine) sebelum evaluate window
- Reset oleh: F-09 (saat posisi closed) via event

- State persistence: SQLite (A6)

Pastikan implementasi: Clean Architecture, SOLID Principle, Modular, Production Ready

F-15 · RPC FALLBACK PRIMARY/SECONDARY (B4)

Fallback Otomatis ke RPC Cadangan

Single point of failure pada satu RPC publik diatasi dengan konfigurasi primary+secondary. Jika primary gagal reconnect (F-01), sistem otomatis beralih ke secondary. Jika keduanya gagal, sistem masuk 'degraded mode' — stop trigger baru tapi tetap pantau posisi terbuka via polling lambat. Tidak ada biaya tambahan — hanya menggunakan dua provider RPC publik gratis yang berbeda.

PROMPT AGENTIC AI — F-15

Anda adalah Senior Software Engineer. Bangun fitur RPC FALLBACK PRIMARY/SECONDARY (Enhancement B4).

Tujuan fitur:

Mengeliminasi single point of failure pada RPC provider dengan konfigurasi primary/secondary yang otomatis switch saat primary gagal, dan degraded mode saat keduanya tidak tersedia.

Fungsi utama:

- Konfigurasi dua RPC endpoint di config.yaml: `rpc.primary_url` dan `rpc.secondary_url`
- F-01 selalu gunakan primary dalam kondisi normal
- Saat F-01 gagal reconnect ke primary (setelah N retry): emit `RpcFailoverEvent` ke F-15
- F-15 switch koneksi F-01 ke secondary: buka WebSocket baru ke secondary endpoint
- Log switchover event dengan timestamp ke dashboard (F-07) sebagai system alert
- Jika secondary juga gagal: masuk 'degraded mode':
- * Stop trigger engine (F-03) — tidak ada sinyal baru
- * Switch monitoring posisi terbuka (F-09 Lapis 3) ke polling via REST (lebih lambat, tapi tidak buta)
- * Alert kritis ke dashboard
- Saat primary pulih: kembali ke primary, log pemulihan, exit degraded mode

Validasi:

- Primary dan secondary harus URL yang berbeda (provider berbeda, bukan instance yang sama)
- Health check periodic ke kedua endpoint (setiap 60 detik) untuk deteksi recovery

Kondisi gagal (degraded mode):

- Polling interval untuk Lapis 3 dalam degraded mode: 30 detik (vs real-time WS)
- Monitoring posisi harus tetap berjalan — TIDAK boleh berhenti walau trigger baru di-stop

Integrasi:

- Digunakan oleh: F-01 (Wallet Movement Monitor)
- Trigger: F-01 emit `RpcFailoverEvent` saat reconnect gagal
- Monitoring: F-07 (Dashboard) untuk status RPC dan switchover history
- Konfigurasi: F-18 (`rpc.primary_url`, `rpc.secondary_url`, `rpc.max_retry`)

Pastikan implementasi: Clean Architecture, SOLID Principle, Modular, Production Ready

F-16 · POSITION CORRELATION CAP (B5)

Batas Posisi Bersamaan untuk Hindari Overexposure

Di musim meme coin, beberapa sinyal berkorelasi bisa muncul bersamaan. 1% risk per trade hanya berlaku jika posisi tidak berkorelasi — jika 5 posisi dengan token serupa terbuka bersamaan, exposure riil bisa jauh lebih besar. Fitur ini menetapkan batas N posisi terbuka bersamaan — bukan menetapkan nilai N, tapi menyediakan mekanisme agar batas yang Anda tentukan benar-benar ditegakkan.

PROMPT AGENTIC AI — F-16

Anda adalah Senior Software Engineer. Bangun fitur POSITION CORRELATION CAP (Enhancement B5).

Tujuan fitur:

Membatasi jumlah posisi terbuka bersamaan untuk mencegah overexposure saat banyak sinyal berkorelasi muncul dalam periode pendek (umum saat meme season).

Fungsi utama:

- Dicek oleh F-08 (Auto Trade Execution) sebelum setiap auto-entry
- Query: `SELECT COUNT(*) FROM open_positions WHERE status='open'`
- Jika `count >= max_concurrent_positions` (dari config): BLOCK eksekusi
- Sinyal tetap di-log ke `inference_log` (konsisten dengan filosofi 'tidak ada sinyal dibuang')
- Emit `'position_cap_reached'` event ke F-07 (dashboard info, bukan critical alert)
- Opsional lanjutan (fase 2): grouping posisi berdasarkan waktu launch token / wallet trigger source untuk correlation analysis

Validasi:

- `max_concurrent_positions` dari `config.yaml` — TIDAK hardcode. Nilai ini adalah keputusan risk management pengguna.
- Jika config tidak ada nilai: default ke nilai konservatif (misal 3), log warning bahwa config belum diset
- Thread-safe query (multiple signals bisa datang bersamaan)

Kondisi gagal:

- Database query gagal: fail-safe → BLOCK eksekusi, log error (jangan asumsi `count = 0`)

Audit:

- Setiap kali cap tercapai: log token yang diblokir, jumlah posisi saat ini, timestamp

Integrasi:

- Dicek oleh: F-08 (sebelum entry order)
- Data source: SQLite `open_positions` (A1, A6)
- Notifikasi: F-07 (Dashboard) — info level
- Konfigurasi: F-18 (`risk.max_concurrent_positions`)

Pastikan implementasi: Clean Architecture, SOLID Principle, Modular, Production Ready

FITUR 17-19

CROSS-LAYER INFRASTRUCTURE

State Persistence, Config Management, Error & Recovery Flow

F-17 · STATE PERSISTENCE & CRASH RECOVERY (A6)

Ketahanan Sistem Saat Restart atau Crash

Sistem berjalan di laptop biasa — bukan server dengan uptime terjamin. Fitur ini memastikan bahwa saat laptop sleep, restart, atau proses crash, sistem bisa pulih ke state yang benar: posisi terbuka ter-restore, proteksi aktif kembali, dan tidak ada posisi yang 'telanjang' tanpa SL/TP/kill-switch aktif.

PROMPT AGENTIC AI — F-17

Anda adalah Senior Software Engineer. Bangun fitur STATE PERSISTENCE & CRASH RECOVERY (A6).

Tujuan fitur:

Memastikan sistem dapat pulih sepenuhnya dari restart, sleep, atau crash dengan me-restore semua state kritis ke kondisi terakhir yang valid — terutama posisi trading yang sedang terbuka.

Fungsi utama:

PERSISTENCE (tulis ke disk):

- open_positions table di SQLite: ditulis SEGERA setelah order entry confirmed (bukan di-batch)

Fields: token, entry_price, size_usd, sl_level, trailing_tp_state, entry_timestamp, model_version, status='open'

- model_registry: model aktif dan versinya (untuk F-05)

- watchlist_wallets: termasuk auto-discovered (untuk F-01)

- retrain_timestamp: waktu retrain terakhir (untuk F-10 catch-up)

- cooldown_state: per (wallet, token) (untuk F-14)

RECOVERY (baca saat startup):

1. Load watchlist_wallets aktif → F-01 subscribe ke semua

2. Load model aktif dari model_registry → F-05

3. Cek retrain_timestamp → jika > 24h, trigger F-10 catch-up

4. Query open_positions WHERE status='open' → untuk setiap posisi:

- a. Fetch current price dari RPC/DexScreener

- b. Bandingkan dengan SL level: jika price sudah di bawah SL → execute exit market order segera

- c. Jika price masih aman: re-aktivasi F-09 (three-layer protection) untuk posisi ini

5. Load cooldown_state → F-14

WATCHDOG:

- Proses terpisah yang monitor apakah main process berjalan

- Jika main process crash: auto-restart dengan delay 10 detik

- Log semua crash dengan stack trace

Validasi:

- open_positions write harus atomic (transaction SQLite)

- Jika database corrupt: failover ke backup database (backup harian), alert kritis ke dashboard

- Startup recovery harus selesai SEBELUM mulai menerima event baru dari F-01

Kondisi gagal:

- Database tidak bisa dibuka saat startup: HALT sistem, alert ke user dengan instruksi recovery manual
- open_positions inconsistent (harga tidak bisa di-fetch): tandai posisi sebagai 'recovery_failed', alert kritis

Audit:

- Setiap startup: log startup_time, open_positions_count, recovery_actions
- Setiap crash yang terdeteksi watchdog: log crash_time, restart_count

Integrasi:

- Menyediakan foundation untuk: F-01, F-05, F-09, F-10, F-14
- Database: SQLite tunggal (closed_trades + open_positions + watchlist + model_registry)

Pastikan implementasi: Clean Architecture, SOLID Principle, Modular, Production Ready

F-18 · CENTRALIZED CONFIG MANAGEMENT (A4)

Manajemen Konfigurasi Terpusat

Semua angka threshold, parameter model, dan pengaturan operasional disimpan di satu file config.yaml. Tidak ada angka yang hardcode di dalam kode. Ini memudahkan kalibrasi ulang tanpa menyentuh kode, dan memungkinkan backtesting dengan kombinasi parameter berbeda.

PROMPT AGENTIC AI — F-18

Anda adalah Senior Software Engineer. Bangun fitur CENTRALIZED CONFIG MANAGEMENT (A4).

Tujuan fitur:

Menyediakan single source of truth untuk semua parameter operasional sistem melalui config.yaml, dengan validasi saat startup dan hot-reload tanpa restart untuk parameter non-kritis.

Fungsi utama:

- Parse dan validasi config.yaml saat startup
- Expose typed config object yang digunakan oleh semua komponen (dependency injection)
- Validasi: setiap field memiliki expected type, range, dan default value
- Hot-reload: watch config.yaml untuk perubahan, reload parameter non-kritis tanpa restart (kritis = model params, RPC URL → butuh restart; non-kritis = threshold = hot-reload aman)
- Config versioning: simpan timestamp perubahan config

Structure config.yaml (sesuai A4):

```
trigger_engine: {window_minutes, mode}
```

```
decision_gate: {confidence_threshold}
```

```
risk: {risk_pct_per_trade, max_concurrent_positions}
```

```
trailing_tp: {tiers: [{r_min, r_max, trail_pct}]}
```

```
labeling: {buy_benar_threshold_r, salah_threshold_r}
```

```
retrain: {schedule_utc, min_closed_trades_first, min_closed_trades_alt, min_buy_benar_in_alt,
rolling_window_days, rollback_accuracy_drop_pct}
```

```
kill_switch: {dev_wallet_sell_threshold_pct, slippage_spike_threshold_pct}
```

```
rpc: {primary_url, secondary_url, max_retry, heartbeat_interval_seconds}
```

```
filter: {min_value_usd, min_token_age_minutes, min_liquidity_usd}
```

```
dex_routers: [list of known DEX router addresses]
```

Validasi saat startup:

- Semua required fields harus ada (fail fast jika tidak)
- window_minutes > 0, confidence_threshold antara 0.0-1.0
- trailing_tp tiers harus berurutan dan tidak overlap
- RPC URL harus valid format HTTP/WS URL

Kondisi gagal:

- config.yaml tidak ada: HALT sistem dengan instruksi pembuatan config
- Field required tidak ada: HALT dengan daftar field yang kurang
- Hot-reload parse error: keep konfigurasi lama, log error, alert ke dashboard

Integrasi: Digunakan oleh SEMUA komponen F-01 sampai F-16. Inject sebagai dependency, bukan global variable.

Pastikan implementasi: Clean Architecture, SOLID Principle, Modular, Production Ready

F-19 · ERROR & RECOVERY FLOW ENGINE (A3)

Mesin Penanganan Error dan Recovery Terpusat

Sistem yang memegang posisi terbuka dengan uang riil tidak boleh diam saat terjadi kegagalan. Fitur ini mendefinisikan perilaku yang diharapkan untuk setiap skenario kegagalan kritis: RPC putus, API down, order gagal, crash. Prinsip utama: fail-closed untuk keputusan trading, fail-open untuk monitoring posisi yang sudah terbuka.

PROMPT AGENTIC AI — F-19

Anda adalah Senior Software Engineer. Bangun fitur ERROR & RECOVERY FLOW ENGINE (A3).

Tujuan fitur:

Menyediakan mekanisme penanganan error yang konsisten dan terdokumentasi untuk semua skenario kegagalan kritis, dengan prinsip fail-closed untuk trading dan fail-open untuk monitoring posisi terbuka.

Fungsi utama:

Implementasi recovery behavior untuk setiap skenario (dari A3):

1. RPC WebSocket terputus:

- Exponential backoff reconnect: 1s, 2s, 4s, 8s, 16s (max 5x)

- Jika gagal semua: trigger F-15 (RPC Fallback)

- Alert ke F-07: 'rpc_disconnected'

2. DexScreener/Safety API timeout:

- Fail-closed: block alert, log ke dataset (sinyal tetap tercatat)

- Retry 1x sebelum dianggap gagal (timeout 5 detik per attempt)

- Alert ke F-07: 'safety_api_timeout'

3. pump.fun entry order gagal:

- Retry max 2x dalam < 5 detik

- Jika tetap gagal: log sebagai 'missed_entry' (TIDAK masuk closed_trades)

- Alert ke F-07: 'entry_failed'

4. pump.fun exit order gagal (normal conditions):

- Retry langsung dengan slippage tolerance +0.5% setiap retry

- Alert ke F-07: 'exit_pending' sampai konfirmasi

5. pump.fun exit order gagal (kill-switch active):

- Retry agresif: immediate, no delay, slippage tolerance naik bertahap sampai max

- PRIORITAS MUTLAK: kehilangan ke slippage lebih baik dari tidak bisa exit

- Alert CRITICAL ke F-07

6. Crash saat posisi terbuka:

- F-17 (watchdog) restart proses

- F-17 recovery: re-aktivasi F-09 untuk semua open_positions

7. Retrain terlewat (laptop mati):

- F-17 startup: cek timestamp, trigger F-10 catch-up jika perlu

Error Registry:

- Semua error types terdefinisi sebagai enum dengan severity level (INFO, WARNING, ERROR, CRITICAL)

- Setiap error dicatat: timestamp, error_type, context, recovery_action, resolution_status

- Error log tersimpan di SQLite (structured) bukan plain text

Prinsip implementasi:

- Fail-closed: jika data tidak lengkap → jangan kirim alert / jangan buka posisi

- Fail-open: untuk monitoring posisi terbuka → selalu coba exit sebisa mungkin

- Semua kegagalan tercatat terstruktur (bisa dikorelasikan dengan exit_reason di closed_trades)

Integrasi: Framework error handling yang digunakan oleh F-01, F-06, F-08, F-09, F-10, F-15.

Alert routing ke F-07 (Dashboard) dengan severity-appropriate display.

Pastikan implementasi: Clean Architecture, SOLID Principle, Modular, Production Ready

RINGKASAN FITUR

19 fitur yang diidentifikasi dari Dokumentasi Arsitektur v2 dan Lampiran Teknis, dikelompokkan berdasarkan layer arsitektur dan prioritas implementasi.

F-01	Wallet Movement Monitor	Layer 0	P0 — Core
F-02	Rule-Based Relevance Filter	Layer 0/1	P0 — Core
F-03	Wallet Movement Trigger Engine	Layer 1	P0 — Core
F-04	Feature Extraction Pipeline	Layer 2	P0 — Core
F-05	XGBoost Inference Engine	Layer 2	P0 — Core
F-06	Token Safety Check Gate	Layer 2/3	P0 — Core
F-07	Web Dashboard Alert Engine	Layer 3	P0 — Core
F-08	Auto Trade Execution	Layer 3	P0 — Core
F-09	Three-Layer Position Protection	Layer 3	P0 — Core
F-10	24H Retrain & Rollback Scheduler	Layer 4	P0 — Core
F-11	Model v0 Bootstrap	Layer 4	P0 — Core
F-12	Dynamic Wallet Discovery (B1)	Layer 0 Enhancement	P1 — Enhancement
F-13	Token Age & Liquidity Hard Filter (B2)	Layer 1 Enhancement	P1 — Enhancement
F-14	Dedup & Cooldown per Wallet-Token (B3)	Layer 1 Enhancement	P1 — Enhancement
F-15	RPC Fallback Primary/Secondary (B4)	Layer 0 Enhancement	P1 — Enhancement
F-16	Position Correlation Cap (B5)	Eksekusi Enhancement	P1 — Enhancement
F-17	State Persistence & Crash Recovery (A6)	Cross-Layer Infra	P0 — Core
F-18	Centralized Config Management (A4)	Cross-Layer Infra	P0 — Core
F-19	Error & Recovery Flow Engine (A3)	Cross-Layer Infra	P0 — Core

Dokumen ini adalah output analisis fitur. Setiap Prompt Agentic AI di atas dapat langsung diberikan kepada AI Coding Agent (Claude Code, GitHub Copilot, Cursor, dll) untuk memulai implementasi. Urutan implementasi yang disarankan: F-17 (State) → F-18 (Config) → F-01 → F-02 → F-03 → F-04 → F-05 → F-06 → F-07 → F-11 → F-08 → F-09 → F-10 → F-12 sampai F-16 → F-19.